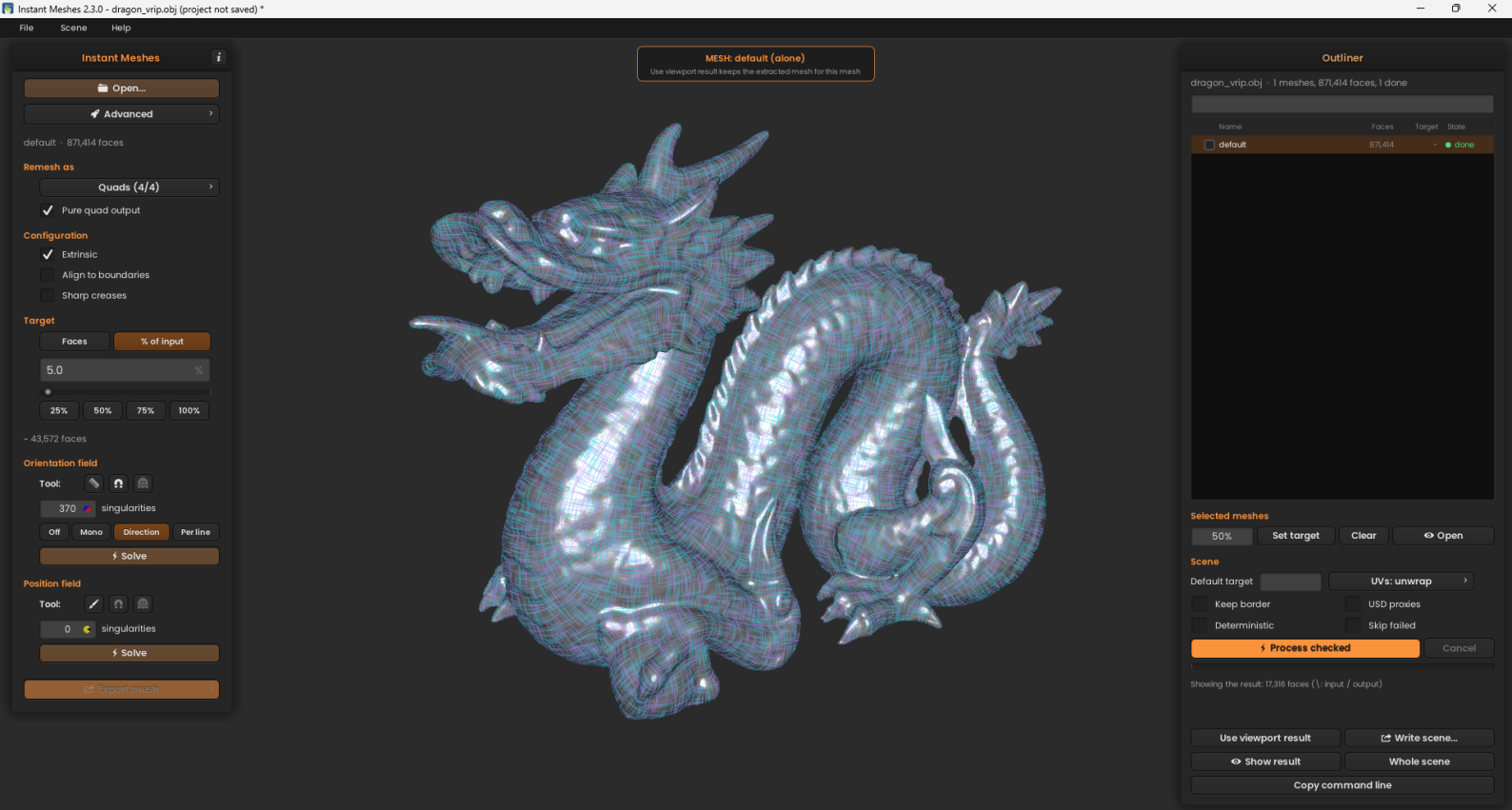




ALEMBIC · RETOPOLOGY · VFX PIPELINE

Instant Meshes

Documentation — MattRM2 fork of Instant Meshes 1.0

Field-aligned automatic retopology with native Alembic and USD support, per-mesh remeshing, percentage targets and the Matt Dark interface.

GitHub : <https://github.com/MattRM2/instant-meshes>

Created by Matthieu Barbié · 2026

Release 2.4.0

Based on Instant Meshes by W. Jakob, M. Tarini, D. Panozzo and O. Sorkine-Hornung (BSD license). Unofficial fork.

0 Table of Contents

1 Overview

- 1.1 What this fork adds
- 1.2 Installation
- 1.3 Files in the release

2 The Matt Dark Interface

- 2.1 Panel reference
- 2.2 Target: faces or percentage
- 2.3 Flow line colors
- 2.4 Typical workflow
- 2.5 Menus and shortcuts
- 2.6 The Outliner: a scene, mesh by mesh
- 2.7 Projects (.imd)

3 Alembic and USD Support

- 3.1 What is read
- 3.2 What is written
- 3.3 Robustness
- 3.4 USD: what is read
- 3.5 USD: the layer written
- 3.6 USD: proxies (--proxy)
- 3.7 From an OBJ to a USD asset with its proxy

4 Command Line

- 4.1 Batch mode
- 4.2 Options
- 4.3 Examples

5 Per-Mesh Remeshing (Alembic, OBJ, USD)

- 5.1 Rules: -m and --others
- 5.2 Planning: --list and --dry-run
- 5.3 What happens to a remeshed mesh
- 5.4 OBJ scenes
- 5.5 Touching objects: --keep-border
- 5.6 Following a long run: --progress

6 UVs

- 6.1 Transfer: --uv transfer
- 6.2 Unwrap: --uv unwrap

7 Reference

7.1 Accuracy of the targets

7.2 Limitations

7.3 Credits and licenses

1

Overview

Automatic retopology, now at home in an Alembic pipeline

Instant Meshes turns dense or messy geometry (scans, sculpts, boolean results) into a clean quad or triangle mesh whose edges follow the shape of the model. This release is a fork of Instant Meshes 1.0 that keeps the original algorithm and its robustness, and adds what a VFX pipeline needs: **native Alembic and USD** input and output, **per-mesh remeshing** inside an Alembic scene, **percentage targets**, and a restyled **Matt Dark** interface.

1.1

What this fork adds

- **Alembic (.abc)** read and written natively: no Alembic library, no dependency, polygon meshes, transforms and instances.
- **USD (.usd, .usda, .usdc, .usdz)** read and written natively, without the USD library, composition included (sublayers, references, payloads, variants, the layers inside a .usdz); per-mesh results written as a **layer** over the original file, which stays untouched.
- **USD proxies** (--proxy): light remeshed copies for the viewports, in a proxy layer that the scene references, as in production; materials and UVs carried over, .usdz packages included.
- **Per-mesh remeshing**: remesh MeshA at 75% and MeshB at 85% of their faces, keep the rest of the scene untouched, write one file (-m / --others), for **Alembic, OBJ and USD** scenes.
- **Percentage targets**: 75% means 75% of the original face count, in the interface and on the command line.
- **UVs**: the original's carried over (--uv transfer), or new ones unwrapped (--uv unwrap).
- **Outliner and projects**: the meshes of a scene in a tree, a target per mesh, processing in the background, and the whole job saved as a project (.imd) that the command line can run.
- **Typed targets**: faces or %, presets and a full-width slider.
- **Meaningful colors**: flow lines by direction (U / V), your strokes in orange, singularities with consistent colors.
- **OBJ reader fixed**: n-gons (faces with more than 4 corners used to be cut silently) and negative (relative) indices.
- **InstantMeshes.exe**, without a space: easier to call from scripts.

1.2

Installation

1

Unzip the release anywhere, for example **D:\Tools\InstantMeshes**.

2

Run **InstantMeshes.exe**. Nothing to install: the executable is self-contained.

3

Optional: add the folder to your **PATH** to call **InstantMeshes** from any terminal or script.

1.3 Files in the release

File	Role
InstantMeshes.exe	The application: interface and command line
InstantMeshes_Documentation.pdf	This document
LICENSE.txt	Instant Meshes BSD license
LICENSE_xatlas.txt	License of xatlas (MIT), used by --uv unwrap
OFL_Poppins.txt	License of the Poppins typeface used by the interface

2

The Matt Dark Interface

Same workflow as Instant Meshes, clearer controls

The panel keeps the original top-to-bottom workflow: open a mesh, choose the output, set a target, solve the orientation field, solve the position field, export. Sections are titled in orange; the main actions (Solve, Extract, Export, Save) are orange buttons.

2.1

Panel reference

Control	What it does
Open...	A scene (.abc, .obj, .usd*: its meshes in the Outliner), a project (.imd), a .ply mesh or an .aln point cloud. Dropping a file on the window opens it too. The line below shows the file and its face count.
Advanced	Saves / loads the session, visualization modes, render layers, hierarchy level, crease angle.
Remesh as	Triangles (6/6), Quads 2/4 or Quads 4/4 (default).
Pure quad output	Subdivides the extracted mesh once so that every face is a quad. Moved here from the Export menu because the target accounts for it.
Configuration	Extrinsic smoothing (recommended), align to open boundaries, keep sharp creases above an angle.
Target	Output density, typed in faces or in % of the input (2.2).
Orientation field	Tools (comb, attractor, repeller), singularity count, flow line colors (2.3) and Solve.
Position field	Tools (edge brush, attractor, repeller), singularity count and Solve.
Export mesh	Smoothing iterations, Extract mesh, Show input / output, Save (.obj, .ply, .abc).

2.2

Target: faces or percentage

Choose **Faces** to type an exact face count, or **% of input** to type a percentage of the polygons of the loaded file (100% = as many faces as the input, 50% = half). Press **Enter** to apply. The line below the controls converts one into the other.

- When a mesh is opened, the target starts at **100%** of its polygons.
- **Presets** 25 / 50 / 75 / 100% apply a percentage in one click.
- The **slider** spans 0 to 100% of the loaded polygons: its right end is the input face count. For more than 100%, type the value. Its **red range** marks targets the input is too coarse for: choosing one proposes to subdivide the input first.
- The target is the **final** face count: with **Pure quad output** the extraction aims at a quarter, the subdivision brings it back to the target.

Accuracy

On real meshes the result lands within about $\pm 3\%$ of the target. Very small meshes (a few hundred faces) can deviate more, since each closed object needs a minimum number of faces.

2.3 Flow line colors

Once the orientation field is solved, flow lines show how the edges of the output will run. Four modes, next to the Solve button:

Mode	Use it to
Off	See only your strokes and the singularities.
Mono	Read the flow without color noise (light grey).
Direction	Tell edge loops apart at a glance: cyan lines follow U, violet lines follow V (green: third direction in triangle mode). Default.
Per line	Follow one loop around the model: one color per line.

Orange is reserved for what you draw. Everything computed uses other colors:

	U flow lines	#22D3EE	Direction mode, first family
	V flow lines	#A78BFA	Direction mode, second family
	W flow lines	#4ADE80	Triangle mode, third family
	Comb strokes	#FB923C	Your orientation comb strokes
	Edge brush strokes	#F5F5F5	Your edge brush strokes
	Valence 3	#60A5FA	Orientation singularity
	Valence 5	#F87171	Orientation singularity
	Position singularity	#FACC15	Small offset
	Position singularity	#F472B6	Large offset
	Input mesh	#6B6B6B	Neutral clay so colors stand out

2.4 Typical workflow

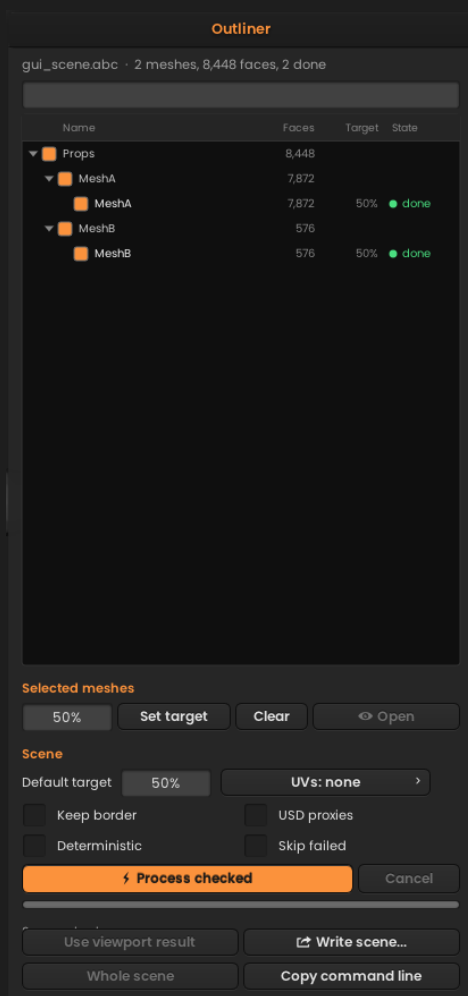
- 1 **Open...** a file and pick **Remesh as** (Quads 4/4 for most models).
- 2 Set the **Target**, for example 50% of the input.
- 3 Click **Solve** under Orientation field. Check the flow in Direction mode; comb the field or move singularities where needed, then stop the solver.
- 4 Click **Solve** under Position field, then open **Export mesh**.
- 5 **Extract mesh**, check it with **Show output**, then **Save** as .obj, .ply, .abc, .usda, .usdc or .usdz.

2.5 Menus and shortcuts

Menu	Entries
File	New (Ctrl+N), Open... (Ctrl+O), Open recent, Save (Ctrl+S), Save as... (Ctrl+Shift+S), Import legacy state..., Export mesh..., Write scene..., Open .imd files with Instant Meshes, Quit (Ctrl+Q)
Scene	Process checked meshes, Cancel processing, Open selected mesh, Show the whole scene, Use the viewport result, Copy the command line, Show the Outliner
Help	About

A * in the window title marks unsaved changes: New, Open and Quit ask before losing them, the window's close button too.

2.6 The Outliner: a scene, mesh by mesh



Opening a scene lists its meshes on the right, as a tree of their paths. The viewport shows the whole scene; a **double click** on a mesh opens it alone, with the interactive tools.

- **Check** the meshes to process (a group checks its meshes). Click selects, Ctrl adds, Shift extends; the header sorts by name, faces or state; the filter keeps the matching paths (Rock, *Rock*, Props/*).
- **Set target** gives the selected meshes their own target (like -m), in orange; the others take the **Default target** (--others), muted. Clear goes back to it.
- **Scene** settings are those of the command line: UVs, keep border, USD proxies, deterministic, skip failed; Remesh as, configuration and smoothing come from the left panel.
- **Process checked** remeshes in the background (progress bar, Cancel); each mesh turns **done**, **failed** (the scene keeps it unchanged) or **stale** (its target or the scene changed).
- A mesh worked by hand: **Extract mesh**, then **Use viewport result**. The banner at the top of the viewport says what it shows: **WHOLE SCENE** (all meshes merged into one) or **MESH: name (alone)**. Extracted from the whole scene, the result is split back into its meshes (each face goes to the nearest mesh); extracted from one mesh, it stays with it.
- **Write scene...**: an Alembic or OBJ copy, or a USD layer, with every done mesh. **Copy command line**: the same job for the render farm.

2.7 Projects (.imd)

File > Save writes the whole job as a project: the scene (referenced, not copied), the settings, every mesh's target and state, the results already computed and the brush strokes of the meshes worked by hand. Reopening it computes nothing again.

- If the scene moved, the project looks next to itself, then asks where it is. If the scene changed, the meshes whose geometry changed are marked stale.
- The command line runs a project (farm) and the interface opens one made by the command line: **--save-imd job.imd** saves a plan, **InstantMeshes.exe job.imd -o scene_retopo.abc** runs it (section 4).
- Binary chunks with a checksum each, results compressed (LZ4), settings in readable text; a damaged project is refused, never half read.
- **File > Open .imd files with Instant Meshes** (or **--register-imd**) opens projects with a double click, with their own icon (Windows, for your user account, no administrator rights).

3

Alembic and USD Support

Native readers and writers, no Alembic or USD library

Alembic files are read and written by code written for this fork, on top of the Ogawa container used by every current DCC. The files it writes open in Blender (and any Alembic reader) exactly like those written by the official library, down to the content hashes readers use for caching.

3.1

What is read

- Every **polygon mesh** of the file, placed in world space through its parent transforms (translate, rotate, scale, matrix, non-inheriting transforms).
- **Instances**: each instance appears at its own place.
- **N-gons** are triangulated for solving; quads keep the split of the OBJ reader, so an .abc and the same .obj give the same triangles.
- Animated files are read at their **first frame**.
- Cameras, curves, points and face sets are ignored; subdivision surfaces are skipped with a message.

3.2

What is written

- A new file: one transform and one polygon mesh named after the file, with quads and n-gons as extracted (no triangulation).
- In **per-mesh mode** (section 5): a copy of the input where only the chosen meshes change.
- The output may be the input file: it is fully read, then replaced in one atomic step. If anything fails, the original stays untouched.

3.3

Robustness

Every position, size and count read from a file is checked before use; nesting depth, cycles and instancing loops are bounded. A damaged file gives an error message, never a crash.

Tested

More than 31,000 deliberately corrupted files are read in the test suite without a single crash. Legacy HDF5-based Alembic files are refused with a clear message (re-export them with the Ogawa backend).

3.4

USD: what is read

USD layers are read by code written for this fork too, in their three encodings: text (**.usda**), binary Crate (**.usdc**, every version from 0.2 to the current 0.13, compressed or not) and package (**.usdz**); a **.usd** file is recognized by its content. The file is **composed** into a stage, as USD does: its sublayers, and for every prim its references, payloads, inherits, specializes and selected variants, strongest opinion first; the layers a package holds are read in place (scene.usdz[parts/asset.usdc]). Everything is checked against Pixar's USD library: the same scenes, in the three encodings, read and compose value for value as USD does.

- Every **Mesh** prim defined in the file (def), placed in world space through xformOpOrder: translate, rotate (every axis order), orient, scale, transform, !invert! and !resetXformStack!.
- Animated transforms and points are read at their **first time sample**; left-handed meshes are turned right-handed.
- **UV primvars** (texCoord2f, or float2 named st / uv), indexed or not, faceVarying, vertex or uniform.
- Classes, overs and inactive prims are skipped; meshes below an instanceable prim or a PointInstancer are marked as instanced (listed, not remeshed). Transforms count on Xformable prims only, as in USD.
- A mesh that comes from a reference or a payload can be remeshed or get a proxy like any other: the layer written (or the proxy layer) puts its opinions over it, the referenced files stay untouched.

3.5 USD: the layer written

In per-mesh mode, the result is not a copy of the scene but a light **layer** (.usda text or .usdc binary) that loads the original as a sublayer and overrides only the remeshed meshes. Open the layer in Blender, Houdini, Maya or usdview to see the scene with the new meshes; the original file is never modified.

```
InstantMeshes.exe asset.usdc -o asset_retopo.usda -m "Hero*=50%" --uv transfer
#usda 1.0 (asset_retopo.usda, simplified)
( subLayers = [ @./asset.usdc@ ] upAxis = "Y" metersPerUnit = 0.01 )
over "World" { over "geo" { over "Hero" {
    int[] faceVertexCounts = [4, 4, 4, ...]
    point3f[] points = [...]
    normal3f[] normals = None
    texCoord2f[] primvars:st = [...] (interpolation = "faceVarying")
} } }
```

- Points are written in the mesh's own space: its transforms, material, purpose and other properties come from the original.
- Normals, non-constant primvars (colors...), creases, corners and holes are **blocked** (= None): they no longer match the topology. UVs are written again when --uv is given.
- **GeomSubsets** (per-face materials) are deactivated; the mesh is bound to the most used of their materials.
- The stage metadata (upAxis, metersPerUnit, defaultPrim, frame range) is copied. The layer is a .usda or a .usdc with another name than the input (a .usdz package would have to hold the scene too); the sublayer path is relative when both sit in the same folder.
- **.usdc** is written as Crate 0.8.0 (read by every USD version since 2018), **.usdz** as a standard package; Pixar's compliance checker reports no error on them. Binary files are about 2.5x smaller than text.
- Whole-file mode reads USD like any mesh and can write a standalone **.usda**, **.usdc** or **.usdz** stage: in meters (metersPerUnit = 1) from an OBJ, PLY or Alembic file, as Blender reads them; with the units and up axis of the input from a USD file.

3.6 USD: proxies (--proxy)

With **--proxy**, **-m** and **--others** keep the meshes of the scene and add their remeshed copy as a **proxy**: the light mesh viewports show while the renderer keeps the original. This is the USD purpose mechanism, understood by usdview, Houdini / Solaris, Maya and Blender.

The proxies go to a **proxy layer** of their own, and the scene **references** it, as in production: you keep opening the scene, which now brings its proxies along. Three runs give three proxy layers, all referenced by the one scene.

```
# -o names the scene: it is edited in place
InstantMeshes.exe asset.usdc -o asset.usdc --proxy --others 5%
# another name: a copy of the scene, like a Save As (the original untouched)
InstantMeshes.exe asset.usdc -o asset_v2.usdc --proxy -m "Hero=10%" --others 3%

#usda 1.0 (asset.usda after --proxy, simplified)
def Xform "Asset" (
    prepend references = @./asset_proxy.usda@</Asset>    # the only change
)
{ ... }
```

- In place, the reference is the **only change** to the scene: in a .usdc file the new data is appended, every other byte stays where it was; in a .usda file the reference is written into the text. A copy keeps the format of the scene (.usda, .usdc, or .usd).
- A **.usdz package** stays a package: its root layer gets the reference, the proxy layer goes inside it (asset_proxy.usdc, next to the root layer), its other files (sublayers, textures) are kept as they were. In place or as a copy (-o another .usdz); Pixar's compliance checker reports no error.
- The proxy layer is **<output>_proxy**, next to the output, in its format: asset_proxy.usdc. It holds the proxies and what the meshes gain (purpose, proxyPrim), and loads nothing itself. A second run writes asset_proxy2.usdc, referenced too. The reference goes on the root prim (/Asset).
- The original gets purpose **render** and a **proxyPrim** relationship to its proxy; the proxy is a new Mesh with purpose **proxy**, no subdivision and the material of the original (the most used one of its GeomSubsets). A material outside the root prim (/materials, as Solaris makes them) is bound through a stand-in, /World/proxy_materials/<name>, that **references** it: the same network and textures, its edits followed.
- The proxy gets the **UVs of the original** (--uv transfer is the default with --proxy; --uv none or --uv unwrap change it).
- Assets under the **geo/render** convention get their proxies in **geo/proxy**, same hierarchy: /Asset/geo/render/Body gets /Asset/geo/proxy/Body. The new prims copy the transforms of those they mirror, animation included, so the proxies follow. Never below the render scope: some importers (Blender) skip it whole when proxies are asked for.
- Elsewhere, the proxy is a sibling named **<name>_proxy**, with the transform of the mesh.
- Proxy and guide meshes get no proxy (--list shows the purpose); a proxy that already exists is an error, found before any computation (to redo it, remove its reference). A mesh at the root of the scene (/Hero) cannot get one: it must be under a root prim (/World/Hero). --dry-run shows where each proxy will go and the layer that holds them.
- Blender imports the render meshes by default: tick **Proxy** in the USD import options to see the proxies.

3.7 From an OBJ to a USD asset with its proxy

Two commands: the first converts the OBJ to USD at its own polygon count (100%: remeshed, about as many faces) with new UVs; the second adds a 10% proxy, which takes over those UVs.

```
InstantMeshes.exe model.obj -o model.usda -f 100% --uv unwrap  
InstantMeshes.exe model.usda -o model.usda --proxy --others 10%
```

Open **model.usda**: it references model_proxy.usda, which adds /model/model_proxy. The first command merges every object of the OBJ into one mesh (whole-file mode).

4

Command Line

Batch remeshing from scripts, farms and DCC tools

Give an output file with **-o** and Instant Meshes runs without a window: it loads, remeshes, writes and exits with code 0 on success. Without **-o** the interface opens on the file. Arguments are checked before any computation, so a typo never costs minutes of remeshing.

4.1

Batch mode

```
InstantMeshes.exe input.abc -o output.abc -f 75%
InstantMeshes.exe input.obj -o output.obj -f 5000 -D
```

4.2

Options

Option	Meaning
-o, --output <file>	Output .obj, .ply, .abc, .usda, .usdc or .usdz (batch mode)
-f, --faces <n> <n>%	Face count, or percentage of the input polygons (75%)
-v <n> / -s <length>	Vertex count / edge length in world units
-D, --dominant	Quad-dominant output (no pure quad subdivision)
-r / -p <n>	Orientation / position symmetry: 4/4 quads (default), 6/6 triangles
-c, --crease <deg>	Keep creases sharper than this angle
-b, --boundaries / --keep-border	Align to open borders / also put them back exactly on the input's (5.5)
-S, --smooth <n>	Smoothing iterations (default 2)
-d / -t <n>	Same result on every run (slower) / number of threads
--uv <mode>	UVs of the output: none (default), transfer or unwrap (section 6)
-m, --mesh <name>=<target>	Per-mesh remeshing of an .abc, .obj or USD scene (section 5)
--others <target>	Target for every other mesh of the scene
--proxy	USD: keep the meshes, add the remeshed copies as proxies (3.6)
--list / --dry-run	List the meshes (--sort asc desc, --top) / print the plan
--skip-failed / --progress	Copy failed meshes unchanged and go on / print the progress
--save-imd <job.imd>	Save the plan (and results) of -m / --others as a project (2.7)
<job.imd> -o <scene>	Run a project: compute what is not done, write the scene
--register-imd	Open .imd projects with a double click (Windows, current user)
-V, --version	Print the version

4.3 Examples

```
# Half the faces, pure quads
InstantMeshes.exe scan.obj -o scan_retopo.obj -f 50%
# Triangles, exact count, sharp edges kept
InstantMeshes.exe part.abc -o part_tri.abc -r 6 -p 6 -f 20000 -c 30
# Replace the input file in place
InstantMeshes.exe scene.abc -o scene.abc -f 60%
# A USD asset: a layer over it, the original untouched
InstantMeshes.exe asset.usdz -o asset_retopo.usda -m "Hero=40%" --uv transfer
# Viewport proxies at 5% for every mesh of a USD asset (asset_proxy.usdc)
InstantMeshes.exe asset.usdc -o asset.usdc --proxy --others 5%
# Prepare a job, run it later on the farm (or open it in the interface)
InstantMeshes.exe scene.abc -m "Hero=30%" --others 10% --save-imd job.imd --dry-run
InstantMeshes.exe job.imd -o scene_retopo.abc
```

5

Per-Mesh Remeshing (Alembic, OBJ, USD)

Remesh some objects, keep the rest of the scene

With **-m** and **--others**, each chosen mesh of an Alembic, OBJ or USD scene is remeshed on its own, with its own target, and written back into a copy of the scene, in the same format (USD: into a .usda or .usdc layer over the scene, see 3.5). Everything else, other objects, cameras, curves, animation and metadata, is copied untouched.

```
InstantMeshes.exe scene.abc -o scene_retopo.abc -m "MeshA=75%" -m "MeshB=85%" --others 25%
```

5.1

Rules: -m and --others

Rule	Selects
-m MeshA=75%	The object named MeshA, anywhere, at 75% of its own faces
-m Props/MeshA=5000	The object at that path from the root, 5000 faces
-m "Mesh*=60%"	Every object whose name starts with Mesh (* = any characters)
-m "Rock_??=50%"	Rock_01, Rock_AB... (? = exactly one character)
-m "Props/*=40%"	The children of Props, and the meshes below them
--others 25%	Every mesh no -m selects; without it they are copied untouched

- A name selects the object and every mesh below it; names are case-sensitive; the leading / is optional.
- When several -m select the same mesh, **the last one wins**: write the general rule first, then the exceptions.
- A rule that matches nothing is an error, found before any computation.
- Quote wildcards ("Mesh*=75%"): Git Bash, Linux and macOS shells would expand them.
- **--sort asc** or **desc** orders --list by face count, **--top n** keeps the first n: --sort desc --top 10 isolates the ten heaviest meshes, --sort asc the smallest ones.

5.2

Planning: --list and --dry-run

```
> InstantMeshes.exe scene.abc --list
Polygon meshes in "scene.abc": 2
  /Props/MeshA/MeshA    7872 faces    7958 vertices
  /Props/MeshB/MeshB     576 faces     576 vertices
> InstantMeshes.exe scene.abc --list --sort desc --top 1
Polygon meshes in "scene.abc": 2, by face count (descending), first 1 shown
  /Props/MeshA/MeshA    7872 faces    7958 vertices
> InstantMeshes.exe scene.abc -m "Mesh*=75%" -m MeshB=85% --dry-run
  /Props/MeshA/MeshA    7872 faces  -> 75% (~5904)    [-m Mesh*=75%]
  /Props/MeshB/MeshB     576 faces  -> 85% (~490)     [-m MeshB=85%]
```

Dry run: nothing computed, nothing written.

5.3 What happens to a remeshed mesh

Kept	Dropped
Name, place in the hierarchy, parent transforms (the mesh is remeshed in world space and written back in its own space)	Normals, per-vertex / per-face attributes and UVs (unless <code>--uv transfer</code>): they do not match the new topology
Object and user properties	Several face sets (per-face materials): they cannot follow the new faces
A single face set (one material), rebuilt over all new faces	

Refused on purpose

Animated meshes and instanced meshes cannot be targets (`-m` reports them; `--others` copies them untouched and says so in the plan). If one mesh fails (e.g. no faces for a tiny target), nothing is written, unless `--skip-failed` is given: the failed meshes are then copied unchanged, listed at the end, and the file is written.

5.4 OBJ scenes

```
InstantMeshes.exe scene.obj --list
InstantMeshes.exe scene.obj -o scene_retopo.obj -m "MeshA=50%" --others 80%
```

- **Objects** are the **o** blocks of the file, or its **g** groups when it has no **o** line (Maya, ZBrush). `--list` shows the names found; paths are `/Name`.
- Untouched objects keep their lines as they are: positions, UVs, normals, materials, groups and comments. Only their face indices are renumbered, since the replaced objects change the vertex count.
- A replaced object keeps its **most used material**: OBJ faces inherit the last usemtl, so an object cannot be left without one. Its normals and smoothing groups are dropped, its UVs too unless `--uv transfer`.
- Vertices shared with an untouched object are kept; the `.mtl` file is not touched.
- The output must be an `.obj`; it may be the input file (atomic replacement).

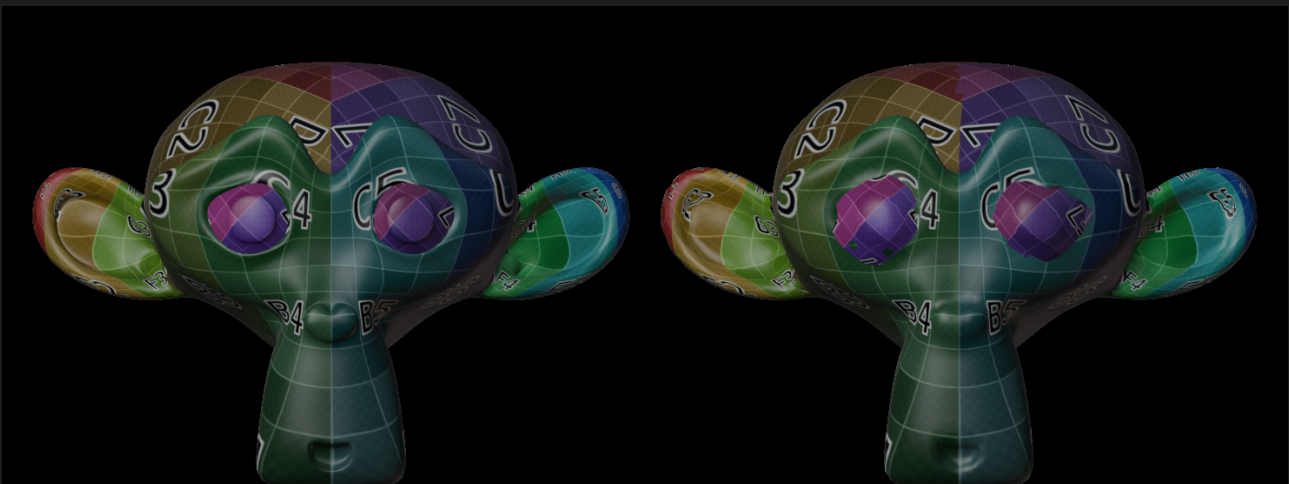
6 UVs

Texture coordinates on the new mesh

A remeshed mesh has a new topology: by default its UVs are dropped. With **--uv transfer**, every UV set of the input is carried over to the new mesh; with **--uv unwrap** the new mesh gets new UVs of its own. Both work in both modes (whole file and -m / --others), for .obj and .abc outputs.

6.1 Transfer: --uv transfer

```
InstantMeshes.exe asset.abc -o asset_retopo.abc --others 25% --uv transfer
InstantMeshes.exe scan.obj -o scan_retopo.obj -f 10% --uv transfer
```



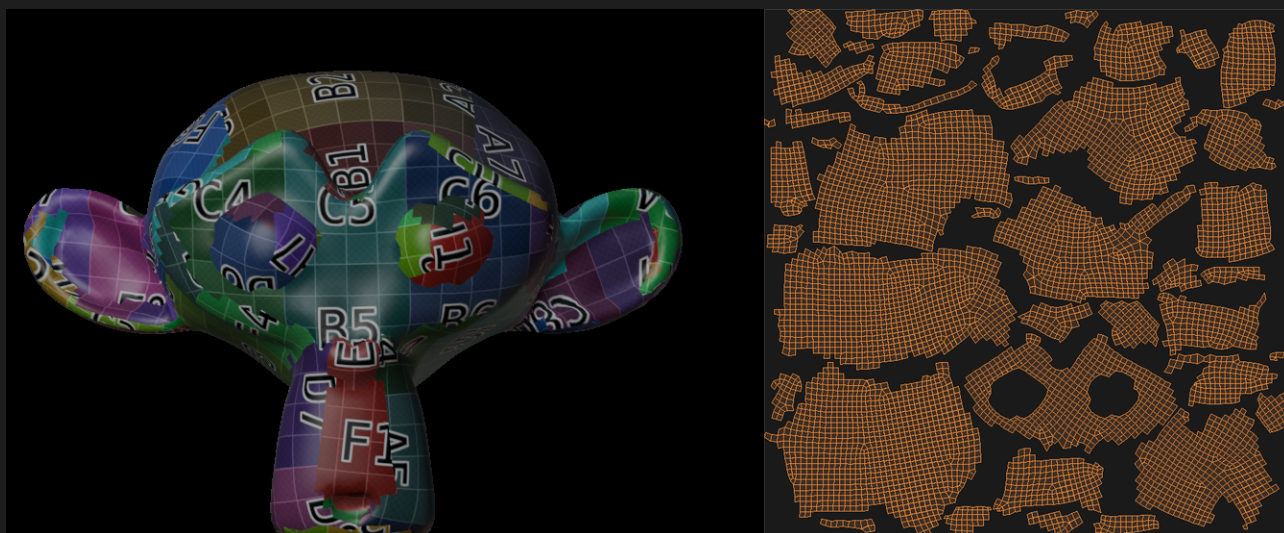
Left: the original (126k faces, Blender UVs). Right: remeshed at 5% with --uv transfer.

- **Island by island:** each new face takes the UV island under its centre, and its corners are projected onto that island only. Past a UV seam the UVs are extended rather than wrapped, so that no face stretches across the texture; the new mesh gets its own seams (per-corner UVs).
- **Every UV set** is kept with its name (UVMap, Planar...): Alembic writes the first one as .geom/uv and the others in .arbGeomParams, as Blender does; an OBJ holds one set (vt).
- Faces keep to the side they face: on a thin wall or a sharp edge, a face never takes the UVs of the opposite side.
- The log tells how many corners were extended past a seam. A mesh without UVs is reported and written without. PLY outputs are refused (no per-corner UVs). Cost: 0.14 s for a 126k-face original.

6.2 Unwrap: --uv unwrap

New UVs for the new mesh, whether the input had UVs or not: the surface is cut into charts, each flattened with little distortion, then packed into the [0, 1] square. One UV set, named UVMMap.

```
InstantMeshes.exe scan.obj -o scan_retopo.obj -f 5% --uv unwrap
```



Remeshed at 5% (6k quads) and unwrapped: the checker keeps one size all over, the quads stay quads in the UV layout (right).

- Computed by **xatlas** (MIT, built in). Every polygon stays whole in one chart (xatlas works on triangles: a quad cut by a chart border is put back together and the charts are packed again).
- No mirrored chart, no overlap, 2 texels of padding at about 1024: ready for baking and painting.
- Cost grows with the face count: about 0.5 s for 6k faces, 4 s for 30k, 11 s for 60k. Unwrap the light meshes (proxies, game assets); transfer the UVs of the heavy ones.

7 Reference

7.1 Accuracy of the targets

Input	Faces	Deviation measured (25 to 150%, quad and -D modes)
Two-object scene	8,448	-1.8% to +1.6%
Open mesh	1,968	-2.6% to +2.0%
Tiny meshes	86	up to $\pm 33\%$

7.2 Limitations

- Alembic: polygon meshes only; subdivision surfaces, curves and points are not remeshed; animation is read at the first frame.
- Remeshed meshes lose their normals (new topology), and their UVs unless `--uv transfer` is given.
- Point clouds (.aln) accept face counts, not percentages.
- USD: layer offsets (time), relocates, value clips and variant fallbacks are not composed; packages inside packages are not opened. A mesh inside an instance cannot be remeshed (edit its prototype asset). Per-mesh output is a .usda or .usdc layer.
- In the interface, a mesh that fails is kept unchanged (as with `--skip-failed`); processing runs one mesh at a time in the background, the viewport shows the scene as one merged mesh.
- **Memory**, per-mesh mode: the remeshing itself takes about 800 bytes per input vertex, for one mesh at a time (the peak is that of the largest remeshed mesh, not of the scene). An Alembic file is read on demand; an OBJ scene is parsed once and kept at about 3x its file size. Remeshed meshes wait in a temporary file next to the output (.spool.tmp, removed at the end).
- A mesh whose longest edges are much longer than the target edge length is subdivided before remeshing (heavier): the log warns about it and lists those meshes at the end.

7.3 Credits and licenses

Instant Meshes: Wenzel Jakob, Marco Tarini, Daniele Panozzo, Olga Sorkine-Hornung, Instant Field-Aligned Meshes, ACM Transactions on Graphics (SIGGRAPH Asia 2015). BSD license, see LICENSE.txt.

This fork (Alembic and USD support, per-mesh remeshing, percentage targets, Matt Dark interface): Matthieu Barbi , 2026, same license. Source: <https://github.com/MattRM2/instant-meshes>

Poppins typeface (interface): The Poppins Project Authors, SIL Open Font License 1.1, see OFL.txt. Hash algorithms: MurmurHash3 (Austin Appleby) and SpookyHash (Bob Jenkins), public domain. UV unwrapping: xatlas (Jonathan Young), MIT license, see LICENSE_xatlas.txt.